Lost in Conversation: Comparing Memory Strategies in Multi-Turn Conversations

Mamitha Muraleedharan
LT2319 Dialogue Systems 2
GitHub

1 Introduction

1.1 Background and Motivation

Today, users interact with large language models (LLM) through multi turn conversations. Rather than providing all their requirements at once, users often share information gradually. Users may also change their preferences or correct earlier information during the conversation.The system needs to remember the latest requirements of the user. In long conversations, managing this information can become quite challenging as information can be missed or incorrectly updated. Previous research by Laban et al. [1] has shown that large language models can struggle in multi turn conversations, particularly when information is provided across several turns. Different approaches can be used to retain user requirements and these approaches may affect how accurately information is retained. In this project, I explore how the different memory approaches behave. I chose Travel Planning as the domain to have a controlled environment for my research as users specify and revise requirements such as destination , budget , travel dates etc.

1.2 Research Question and Hypothesis

My main research question of this project is how do different memory strategies affect the retention and correction of user requirements in multi-turn dialogues? The study focuses on two aspects which are information retention and correction handling. Information retention checks whether the system remembers requirements mentioned earlier and correction handling checks whether it remembers the latest value when a user changes a requirement. To investigate this, three memory strategies are compared and evaluated : Full History,Rolling Summary, SISU. The initial hypothesis was that SISU might be more reliable for tracking corrections than relying only on unstructured conversation history.

2 Data Collection

2.1 Scenario and Datasets

Travel planning was chosen as the domain to have a controlled evaluation. User provide travel requirements such as destination,budget,dates,duration,interests,transport and number of travelers when having conversation with an AI Assistant. During these conversations, users might also change or correct these requirements. Three travel-planning dialogues were used , each conversation focusing on a trip to different destination. Two of the datasets were synthetic dialogues. In this, Paris tests a shorter correction scenario and Rome tests a longer dialogue with multiple corrections. The third dialogue set named Spain was based on a human-AI travel planning conversation which contains more hesitations, fragmented utterances and corrections.The dialogues

were stored as structured TypeScript objects.Each turn contains the speaker (user or assistant) and the message. All three memory strategies are evaluated using the same dialogue set.

Table 1: Dataset Statistics

Dataset	Turns	Description
Paris	19	Short synthetic dialogue
Rome	50	Long synthetic dialogue
Spain	79	Human–AI conversation

2.2 Ground Truth Preparation

The experiment required ground truth data to evaluate whether the strategies retained the correct information. Hence, the expected final travel requirements for each dialogue were constructed. The final requirements were manually defined after considering the user's corrections. The requirements consist of seven fields which are : destination, start date, duration, budget, interests, transport, and travelers.These expected values were used to compare each strategy's predicted requirements against the expected values and calculate accuracy.

3 Implementation

3.1 Full History

The basic idea of this approach is that the entire conversation is retained. Each dialogue turn is added to a history array in chronological order. Each turn contains the speaker and the message. At the end, the complete history is converted into messages and passed to Llama 3.1 through Ollama. The LLM extracts the final seven travel requirements in a structured JSON format. The prompt is given such that the LLM must identify the latest requirements by considering earlier statements and subsequent corrections.

3.2 Rolling Summary Memory

Instead of using the complete dialogue for final extraction, this approach maintains a continuously updated text summary. The summary is initialized as an empty string. Each dialogue is then processed in batches of two turns which will be a pair of user-assistant conversation exchange. Llama 3.1 receives the previous summary and the next two turns. The LLM generates a revised summary, ideally preserving important information and incorporating corrections. At the end, the final summary is passed to Llama 3.1 to extract the seven travel requirements in JSON format.

3.3 SISU-Based Memory

SISU is based on the Information State Update approach to dialogue management. The basic idea is that unlike Full History and Rolling Summary, this approach stores individual facts rather than a text-based memory. A simplified Information State is implemented which contains shared.com and shared.qud. shared.com stores shared commitments like travel requirements and shared.qud tracks Questions Under Discussion. An LLM is used to identify the dialogue moves. For this, each dialogue turn is passed to Llama 3.1. The model receives the current utterance, existing facts, current question and speaker. The model also receives four previous turns so that it can understand the context. It then identifies dialogue moves such as provide, correct, add, ask, and answer.

The approach uses Information State update rules which update the Information State based

on the identified moves. For example, if the budget is changed by the user from 800 to 600, The correct move replaces the previous budget with the new value. QUD helps connect short answers to active questions. At the end, the structured facts are converted directly into the TravelRequirements format. Unlike Full History and Rolling Summary, no additional LLM call is needed for final extraction.

The prompts used for the memory strategies are provided in Appendix B.

4 Evaluation and Results

4.1 Experimental Setup and Evaluation Metrics

All three memory strategies were tested on the same Paris, Rome and Spain datasets. Llama 3.1 was used through Ollama. Preliminary experiments explored temperature settings of 1 and 0. The final evaluation used temperature 0. Each strategy was run five times on each dataset because LLMs may produce different outputs across repeated runs. The final seven predicted travel requirements were compared with the manually defined ground truth. Strict matching was used such that the requirement was counted as correct only when it matched the expected value after basic normalization. Overall requirement accuracy, execution time and consistency across repeated trials were measured.

$$\text{Accuracy} = \frac{\text{Number of correct requirements}}{\text{Total number of requirements}} \times 100$$

4.2 Accuracy and Execution Time

Full history achieved the highest or joint-highest accuracy across all three datasets. Rolling Summary matched Full History on Paris and Rome. SISU had the lowest accuracy across all datasets. All three approaches achieved lower accuracy on Spain. Full History had the lowest execution time across all three datasets. Rolling Summary and SISU took longer, particularly for the Rome and Spain. Also, most strategies produced identical accuracy scores across five trials. Rolling Summary showed some variation on the Spain dataset. One Rolling Summary trial on the Spain dataset failed because the LLM returned an invalid output format. This difference in execution time is partly due to the number of LLM calls made by each strategy.

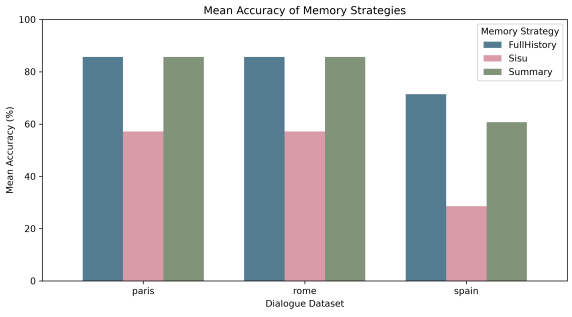


Figure 1: Mean Accuracy across datasets and strategies.

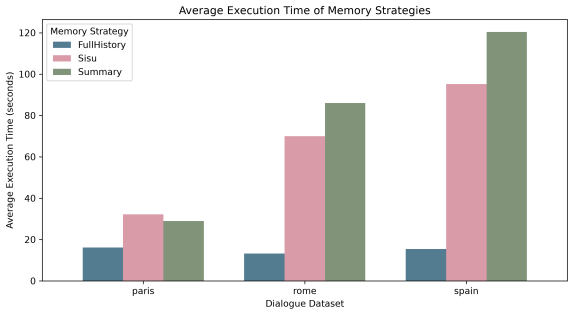


Figure 2: Mean execution time across datasets and strategies.

Table 2: Performance Comparison of Memory Strategies

Dataset	Strategy	Mean Accuracy	Mean Time (s)
Paris	Full History	85.71%	16.2
Paris	Rolling Summary	85.71%	29.4
Paris	SISU	57.14%	32.4
Rome	Full History	85.71%	13.2
Rome	Rolling Summary	85.71%	86.2
Rome	SISU	57.14%	70.0
Spain	Full History	71.43%	15.4
Spain	Rolling Summary	60.71%	120.6
Spain	SISU	28.57%	95.2

5 Discussion and Limitations

Full History performed best overall. Having access to the entire conversation history may have helped the LLM identify the latest requirements and corrections. Rolling Summary sometimes lost or changed information. For example, in Spain, the summary sometimes considered Madrid as the final destination instead of Spain. It also included parks and museums as user interests which were suggestions from the AI assistant, and not something the user agreed. SISU struggled with dialogue move interpretation and QUD handling. For example, it incorrectly stored "with my family" as transport and failed to retain the final budget of €3000. The initial hypothesis was that SISU should handle corrections more reliably. However, our implementation performed worse than the other approaches. The results suggest that structured memory alone is not enough. Dialogue moves must be interpreted correctly, and the update rules must integrate them into the Information State.

The main limitation is that the experiment is limited to three dialogues. This limits how broadly we can generalize the findings. Spain had longer number of turns in conversation and contained more fragmented language and hesitations. Therefore, we cannot conclude that dialogue length alone caused the lower accuracy. A strict matching was performed. Semantically similar expressions may be marked incorrect, such as "history" and "historical places". Another limitation is that each approach used different number of LLM calls. Full History used one final extraction call, while Rolling Summary and SISU made repeated calls. Therefore, execution times are not directly comparable as measures of memory efficiency.

Considering the different memory strategies, each approach had its own limitations. Although Full History performed well in our experiments, processing the entire conversation may become computationally expensive as dialogue length increases. Summary memory sometimes omitted important things when generating summary or hallucinated new information. In SISU approach, the LLM fails to identify the dialogue moves correctly.

6 Future Work

One possible direction for future work is to integrate the memory strategies with SpeechState to test them during live conversations. Three datasets are not sufficient to determine which approach performs best in general. Hence, more natural conversations should be evaluated with different lengths and correction patterns. Future work could also include semantic similarity, token usage, memory efficiency, and separate retention and correction accuracy. The current implementation uses a simplified version of SISU. Dialogue move recognition and QUD handling should be improved so that corrections and short answers are integrated more reliably.

7 Conclusion

The study compared three approaches to maintaining information in multi turn dialogues. Full History achieved the highest or joint-highest accuracy in our experiments. The initial expectation that structured memory would handle corrections more reliably was not supported by the results of this implementation. Accurate interpretation and memory updates are as important as how information is stored.

References

[1] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. Llms get lost in multi-turn conversation. In *International Conference on Learning Representations*, 2026.

A Appendix A : Sample Dialogues and Ground Truth

The following dialogues from the Paris dataset shows how users revise their travel requirements during a conversation.

User: There will be two of us.
User: Our total budget is €800.
User: Actually, three people will be traveling instead of two.
User: Also, our total budget has changed to €600.

Final expected requirements:
Destination: Paris
Start date: 2026-12-10
Duration: 5 days
Budget: 600 EUR
Interests: museums, history
Transport: public transport
Travelers: 3

B Appendix B : LLM Prompts

B.1 Requirement Extraction

The following prompt was used to extract the final travel requirements from Full History and Rolling Summary Memory.

```
Extract the current travel requirements from the conversation.
Return a JSON object containing these fields when mentioned:

destination, startDate, durationDays, budget, interests, transport, travelers.
    Formatting:
        - Dates: YYYY-MM-DD
        - Budget, durationDays, travelers: numbers
        - Interests: array of strings
        - Omit fields that were not mentioned.
        - Return only valid JSON.
```

B.2 Rolling Summary

The following prompt was used to update the summary using the previous summary and new dialogue turns.

```
You maintain a concise rolling summary of a travel-planning
    conversation.

Rules:
    - Preserve important travel requirements.
    - Preserve information from earlier turns unless corrected.
    - When the user corrects a requirement, use the latest value.
    - Preserve destination, dates, duration, budget, interests, transport
    and travelers.
    - Do not invent missing information.
    - Write a concise natural language summary, not JSON.
```

B.3 SISU

The prompt was used to identify ask,answer,provide,add and correct moves.

```
You are the NLU component of a travel-planning dialogue system.
Interpret the utterance as dialogue moves.
    Speaker: ${speaker}
    Supported predicates:destination, startDate, durationDays,
       budget, interests, transport, travelers.
    Supported move types:
        - ask: A question requesting a travel requirement.
        - answer: A short answer to the current question.
        - provide: A new travel requirement.
        - add: Add an additional interest.
        - correct: Explicitly revise an earlier requirement.
    Rules:
        - Use the current question to interpret short answers.
        - Do not treat ordinary questions as new travel requirements.
        - Do not invent information or infer changes that were not
          stated.
        - Only use "correct" when the user explicitly changes an
          existing requirement.
        - Do not interpret assistant acknowledgements as new
          requirements.
        - Use YYYY-MM-DD for dates.
        - Represent proposition arguments as strings.
        - Return {"moves": []} when no relevant dialogue move is
          present.
        - Return only valid JSON.
        - Use "answer" only when a current question is provided.
        - When there is no current question, use "provide", "add",
          or "correct" for travel requirements.
        - Use "provide" when the user states a requirement for the
          first time.
        - Use "correct" when the user revises an existing
          requirement.
```

```
        - Never use "answer" for a standalone requirement when the
          current question is "(none)".
        - For travelers, identify the total number of people
          traveling, including the user.
        - When family members or groups are mentioned, calculate the
          total number of travelers if the information is clear.
        - Use existing traveler information when interpreting
          additions or corrections, and avoid double-counting
          people.
        - Represent the final traveler count as a numeric string
        (e.g., "7", not "seven" or "family").
        - Use recent dialogue context to understand incomplete or
          fragmented utterances.
        - Extract updates only from the current utterance, using
          earlier turns for context. Do not reapply old
          requirements.
    Move formats:
        Ask:
            {"type":"ask","content":{"type":"whq","predicate":"budget"}}
        Answer:
            {"type":"answer","content":"900"}
        Provide, add, correct:
            {"type":"provide","content":{"predicate":"destination",
            "argument":"Rome"}}

Return all moves inside a "moves" array.
```